CEN111
FUNCTIONS I

Introduction

- Most computer programs that solve real-world problems are much larger than the programs presented in the first few chapters.
- Experience has shown that the best way to develop and maintain a large program is to construct it from smaller pieces, each of which is more manageable than the original program.
- This technique is called **divide and conquer**.

Introduction

- **Functions** are used to modularize programs
- C programs are typically written by combining new functions you write with prepackaged functions available in the C standard library.
- The C standard library provides a rich collection of functions for performing common mathematical calculations, string manipulations, character manipulations, input/output, and many other useful operations.

Functions

- We have already written our own functions and used library functions:
  - *main is a function that must exist in every C program.*
  - *printf, scanf are library functions which we have already used in our programs.*
- We can write our own functions to define tasks that may be used at many points in a program.
- These are sometimes referred to as **programmer-defined functions**.

Functions

- We need to do two things with functions:
  - *Create functions*
  - *Call functions (Function invocation)*

- Functions are invoked by a **function call**, which specifies the function name and provides information (as arguments) that the called function needs to perform its designated task.

Function Definiton

A function definition has the following form:

```
return_type function_name (parameter-declarations)
{
    variable-declarations;
    function-statements;
}
```

Function Definiton

- return_type - specifies the type of the function and corresponds to the type of value returned by the function
  - void – indicates that the function returns nothing.
  - if not specified, of type int
- function_name – name of the function being defined (any valid identifier)
- parameter-declarations – specify the types and names of the parameters (a.k.a. formal parameters) of the function, separated by commas.

Function Definiton

- return_type - specifies the type of the function and corresponds to the type of value returned by the function
  - void – indicates that the function returns nothing.
  - if not specified, of type int
- function_name – name of the function being defined (any valid identifier)
- parameter-declarations – specify the types and names of the parameters (a.k.a. formal parameters) of the function, separated by commas.

Example: Function returning a value

```
int cube ( int num )
{
    int result;
    result = num * num * num;
    return result;
}
```

This function can be called as:

n = cube(5);

Example: void Function

```
void message(void)
{
    printf("A message for you:");
    printf("Have a nice day!\n");
}
```

This function can be called as:

message( );

Functions

- All variables defined in function definitions are **local variables**—they can be accessed only in the function in which they're defined.
- Most functions have a list of parameters that provide the means for communicating information between functions.
- A function's parameters are also local variables of that function.

The *return* statement

- When a return statement is executed, the execution of the function is terminated and the program control is immediately passed back to the calling environment.
- If an expression follows the keyword return, the value of the expression is returned to the calling environment as well.
- A return statement can be one of the following two forms:
  - *return;*
  - *return expression;*

The ***return*** statement

- *return;*
- *return 1.5;*
- *return result;*
- *return a+b*c;*
- *return x < y ? x : y;*

Example

```c
int IsLeapYear(int year)
{
    return ( ((year % 4 == 0) && (year % 100 != 0))
        || (year % 400 == 0) );
}
```

This function may be called as:

```
if (IsLeapYear(2005))
    printf("29 days in February.\n");
else
    printf("28 days in February.\n");
```

Example

Input two integers: 5 6
The minimum is 5.

Input two integers: 11 3
The mininum is 3.

```c
#include <stdio.h>
int min(int a, int b)
{
    if (a < b)
        return a;
    else
        return b;
}

int main (void)
{
    int j, k, m;
    printf("Input two integers: ");
    scanf("%d %d", &j, &k);
    m = min(j,k);
    printf("The minimum is %d.\n", m);
    return 0;
}
```

Parameters

- *A function can have zero or more parameters.*
  - *In declaration header:*

    int f (int x, double y, char c);
  - *In function calling:*

    value = f(age, score, initial);

Parameters

- The number of parameters in the actual and formal parameter lists must be consistent
- Parameter association is positional: the first actual parameter matches the first formal parameter, the second matches the second, and so on
- Actual parameters and formal parameters must be of compatible data types
- Actual parameters may be a variable, constant, any expression matching the type of the corresponding formal parameter

Parameters

- Each argument is evaluated, and its value is used locally in place of the corresponding formal parameter.
- If a variable is passed to a function, the stored value of that variable in the calling environment will not be changed.
- In C, all calls are call-by-value unless specified otherwise.

Parameters

Output:

5

0

5

15

```c
#include <stdio.h>
int compute_sum (int n)
{
    int sum;
    sum = 0;
    for (n; n > 0; n--)
        sum += n;
    printf("%d\n", n);
    return sum;
}

int main (void)
{
    int n, sum;
    n = 5;
    printf("%d\n", n);
    sum=compute_sum(n);
    printf("%d\n",n);
    printf("%d\n", sum);
    return 0;
}
```

```c
/* Finding the maximum of three integers */
#include <stdio.h>
int maximum( int x, int y, int z ); //function prototype

int main()
{
    int a, b, c;
    printf( "Enter three integers: " );
    scanf( "%d%d%d", &a, &b, &c );
    printf( "Maximum is: %d\n", maximum( a, b, c ) );
    return 0;
}
//function definition
int maximum( int x, int y, int z )
{
    int max = x;
    if ( y > max )
        max = y;
    if ( z > max )
        max=z;
    return max;
}
```

Function Prototypes

- General form for a function prototype declaration:
  - *return_type* ***function_name*** *(parameter-type-list)****;***
- Used to validate functions
  - *Prototype only needed if function definition comes after use in program*
- The function with the prototype

  int maximum(int, int, int);

  Takes in 3 ints

  Returns an int

Alternative styles for function definition order

```c
#include <stdio.h>

int max (int a, int b)
{
    ...
}

int min (int a, int b)
{
    ...
}

int main(void)
{
  ...
  min(x,y);
  max(x,y);
  ...
}
```

Alternative styles for function definition order

```c
#include <stdio.h>
int max(int,int);
int min(int,int);

int main(void)
{
  ...
  min(x,y);
  max(x,y);
  ...
}

int max (int a, int b)
{
    ...
}

int min (int a, int b)
{
    ...
}
```